

Computer Architecture for Embedded Systems

Student Reference Slides

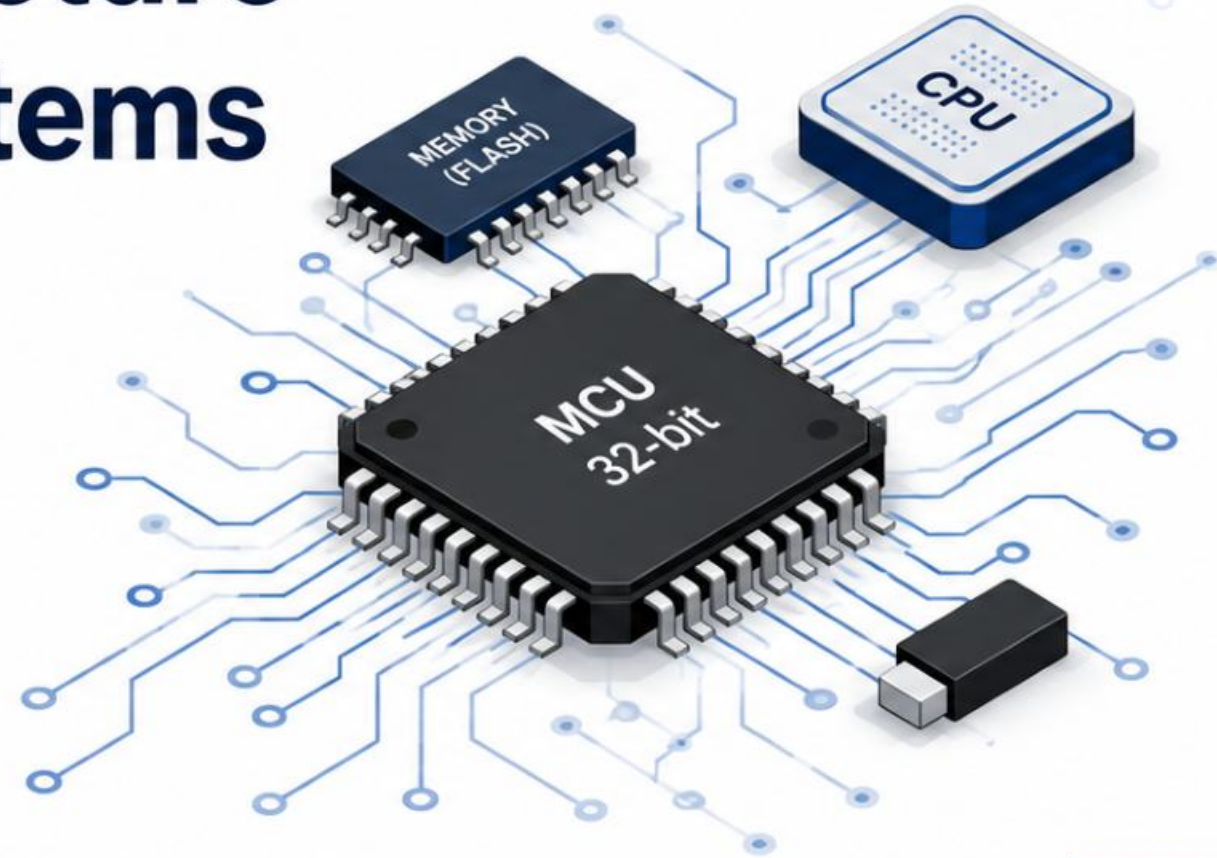


Prepared by Mahmoud Morsy



NTI

National Telecommunication Institute



Embedded Systems and a Real-Life Example

1. What is an Embedded System?

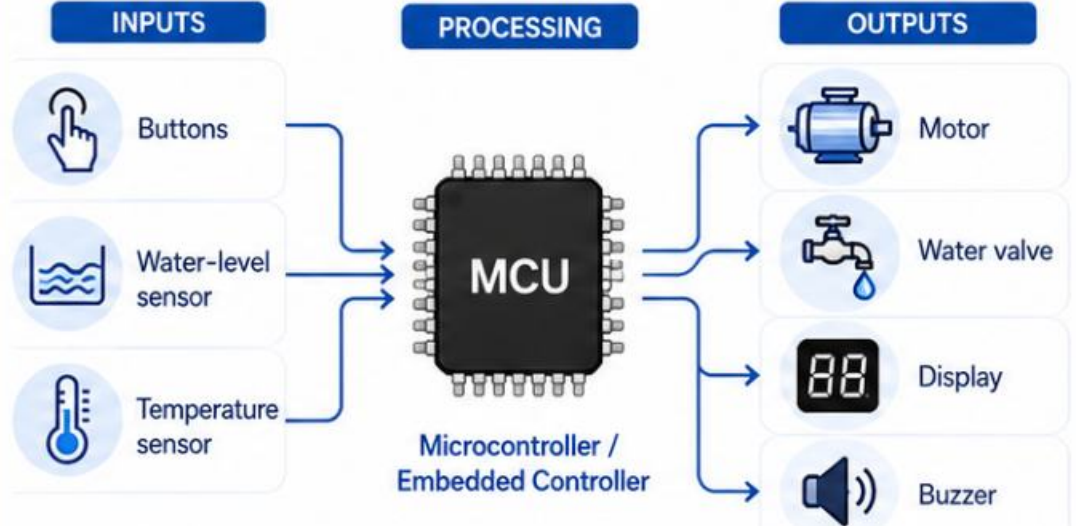
An embedded system is a computer system designed to perform a specific task inside a larger product or machine.

- Dedicated to a specific function
- Combines hardware and software
- Often works in real time
- Found in everyday devices

2. Examples Around Us

-  Washing machine
-  Microwave oven
-  Smart watch
-  Car control unit
-  Printer

3. Real-Life Example: Washing Machine



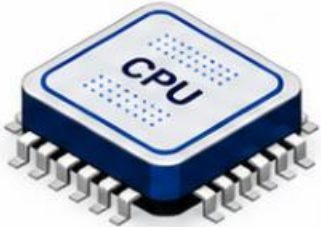
Key Idea

The embedded controller reads inputs, processes data, and controls outputs automatically.

Processor, Microprocessor, and Microcontroller

1 Processor (CPU)

The processor is the core unit that performs calculations, logic operations, and control tasks.



- Executes instructions
- Processes data
- Controls system operations

2 Microprocessor

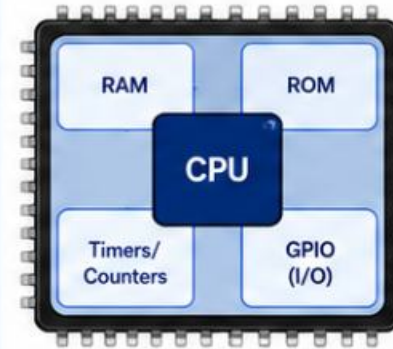
A microprocessor contains mainly the CPU and relies on external memory and peripherals.



- Needs external RAM/ROM
- Used in computers and advanced systems
- Higher flexibility
- Higher system complexity

3 Microcontroller

A microcontroller integrates CPU, memory, and peripherals on one chip.



- Compact solution
- Lower cost
- Lower power
- Widely used in embedded systems

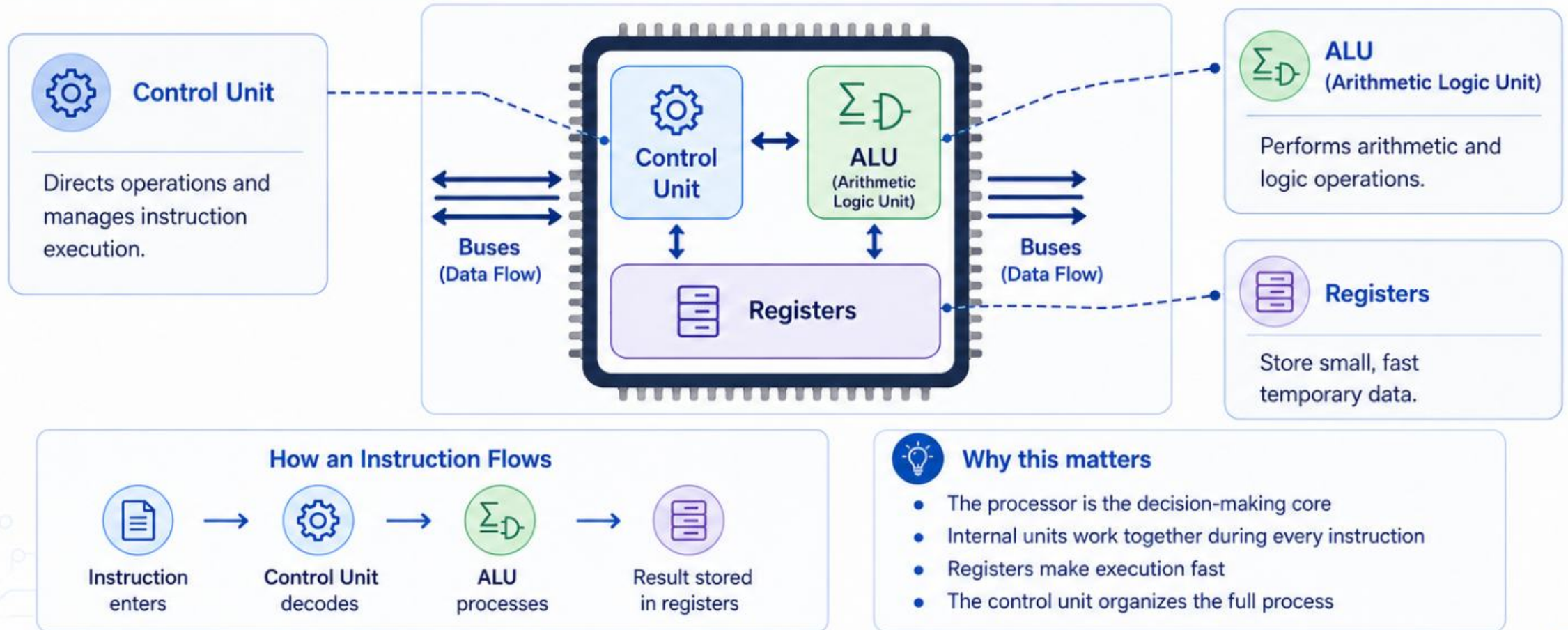
Feature	Processor (CPU)	Microprocessor	Microcontroller
 CPU only	Only the CPU	Only the CPU	CPU + Memory + Peripherals
 Memory location	None	External (off-chip)	Internal (on-chip)
 Peripherals	None	External (off-chip)	Integrated (on-chip)
 Power	N/A	Higher	Lower
 Typical use	The core in any system	Computers, high-performance systems	Embedded control applications, IoT, appliances



Key Difference:

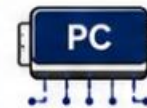
A microprocessor is usually one part of a larger system, while a microcontroller is a complete small control system on one chip.

Inside the Processor



Important Registers in the Processor

Register	Full Name	Main Function
PC	Program Counter	Holds the address of the next instruction
IR	Instruction Register	Holds the current instruction being decoded or executed
ACC	Accumulator	Stores intermediate arithmetic and logic results
PSW	Program Status Word	Stores status flags such as zero, carry, sign, and overflow

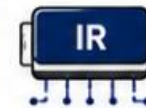


PC

Program Counter



Holds the address of the next instruction.



IR

Instruction Register



Holds the current instruction being decoded or executed.



ACC

Accumulator



Stores intermediate arithmetic and logic results.



PSW

Program Status Word



Stores status flags such as zero, carry, sign, and overflow.

How They Work Together During Execution



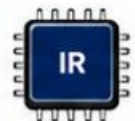
PC

Holds the address of the next instruction.



Fetch Instruction

The instruction is fetched from memory using the address in PC.



IR

Holds the current instruction being decoded.



Decode / Execute

The instruction is decoded and executed.



ACC

Stores the result of the operation.



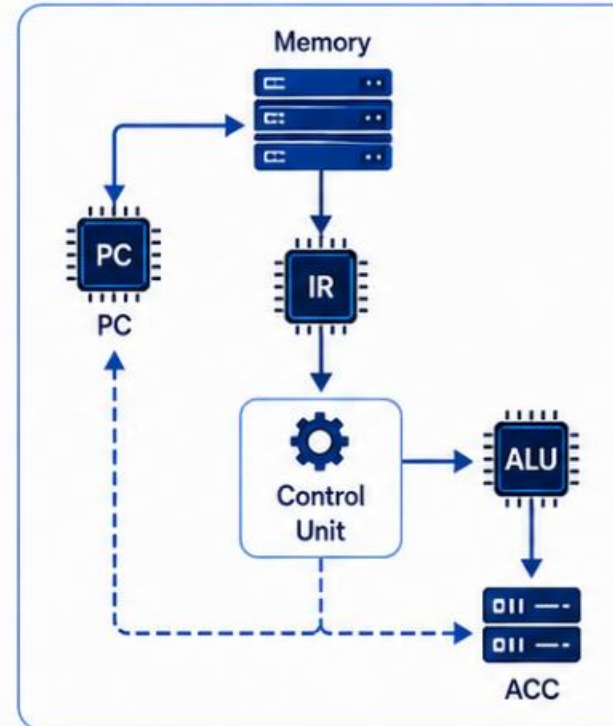
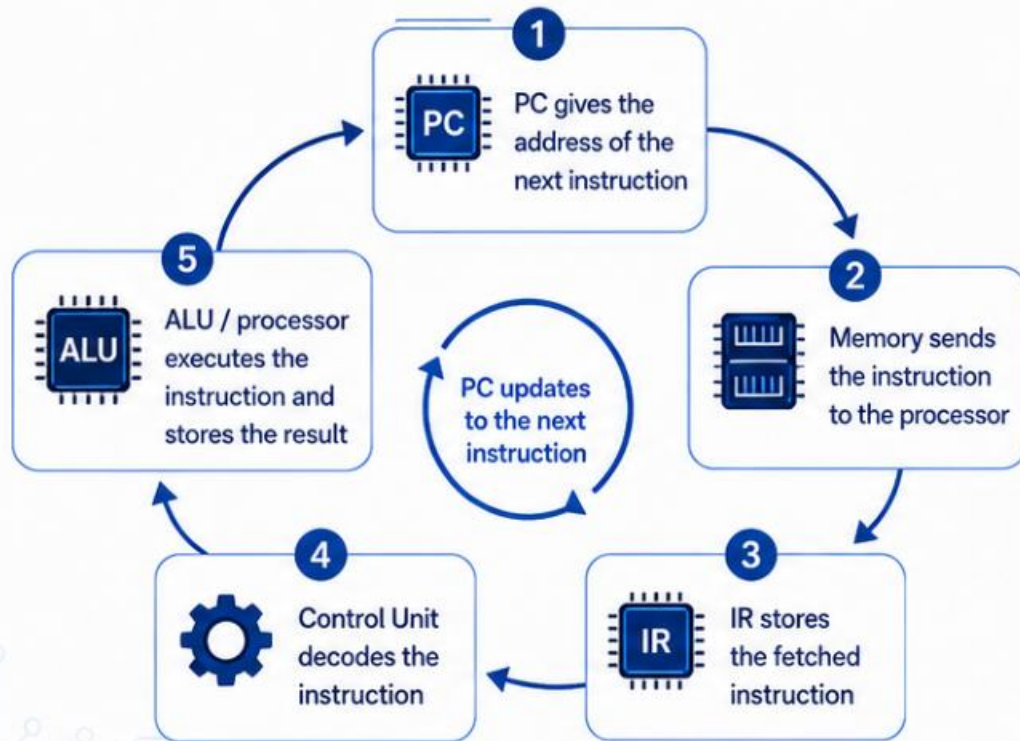
PSW

Flags are updated (based on the result).



Note: Registers are very small, very fast storage locations inside the processor.

Fetch - Decode - Execute Cycle



Worked Example: ADD A, B



Fetch

Get ADD instruction from memory.



Decode

Recognize it as an addition command.



Execute

ALU adds A and B.



Store

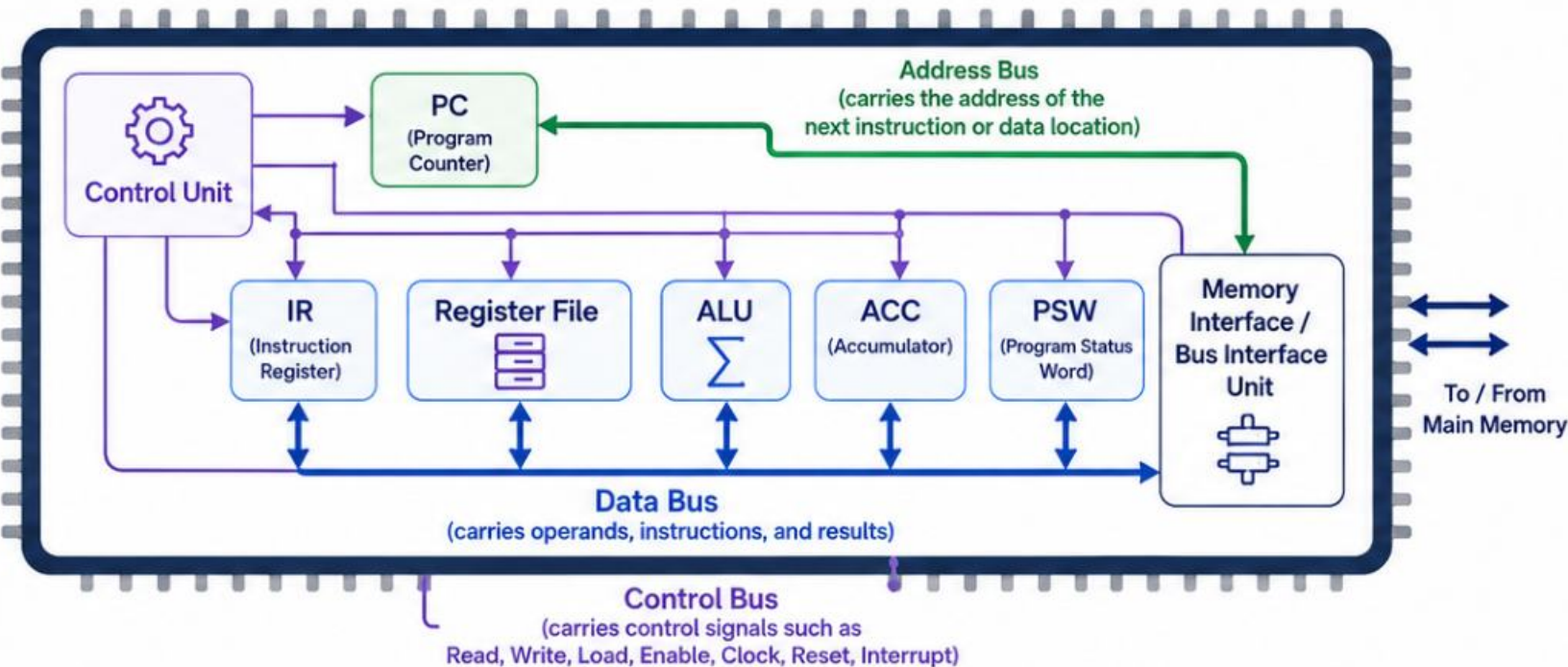
Result goes to ACC or another register.



Summary

This cycle repeats continuously while the processor runs a program.

Processor Internal Buses



Data Bus

Moves the actual binary data inside the processor.

Examples: instruction word to IR, operand to ALU, result to ACC/registers.



Address Bus

Selects where to read/write.

Usually driven by PC or address logic during instruction fetch and memory access.



Control Bus

Tells each unit what to do and when. Enables registers, selects ALU operation, and controls read/write.



Note

Inside the processor, buses are shared communication paths that connect functional units and organize instruction execution.

Example: Fetch and Execute



RISC and CISC Architectures

1. RISC

Reduced Instruction Set Computer

- Small set of simple instructions.
- Commonly uses fixed-length instructions.
- Load/store design: memory access uses LOAD and STORE.
- Simpler decoding; strong fit for pipelining and embedded CPUs.

2. CISC

Complex Instruction Set Computer

- Large set of more complex instructions.
- Often uses variable-length instructions.
- Some instructions combine memory access and arithmetic work.
- More complex decoding and control logic.

Example: adding two values

RISC usually breaks the task into simple steps:

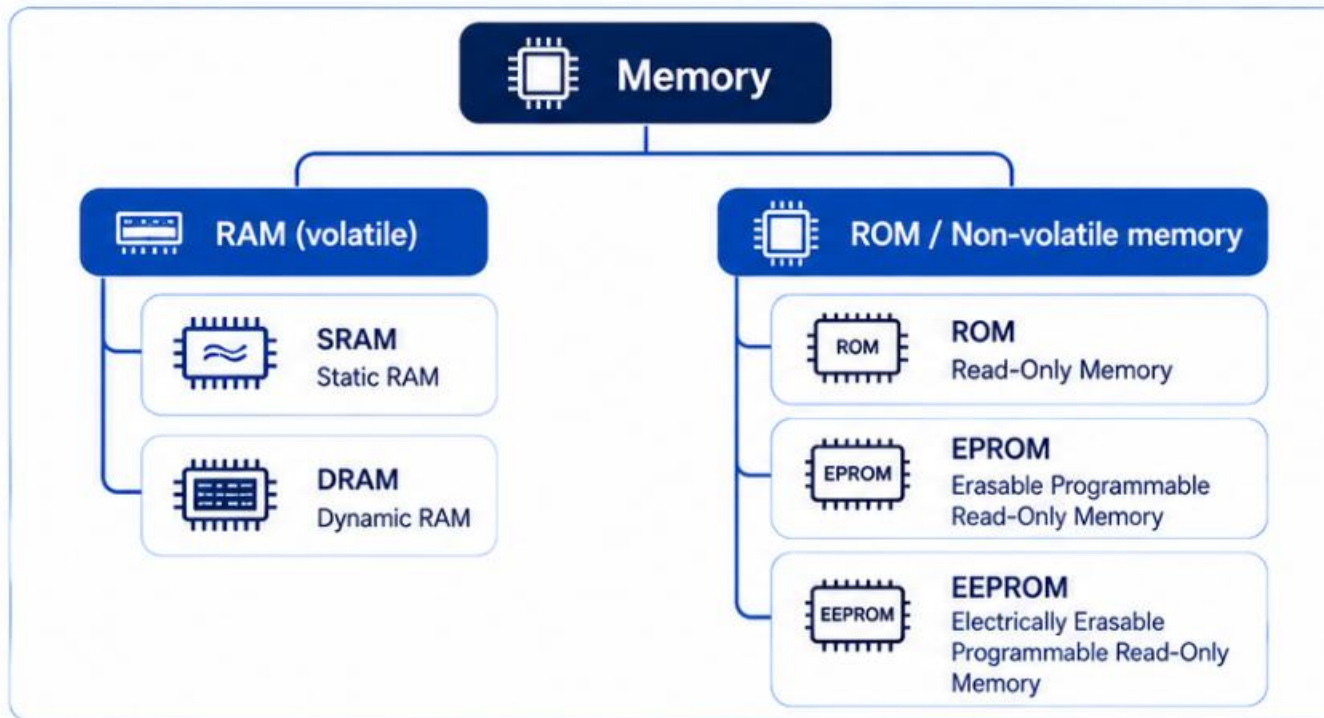
LOAD R1,[A] → LOAD R2,[B] → ADD R3,R1,R2 → STORE [C],R3

CISC can use fewer, more complex instructions:

COMPLEX ADD OPERATION → DECODED TO INTERNAL STEPS

Study note: both styles can run the same program; the difference is the instruction-set philosophy.

Memory Types in Embedded Systems



i Memory Explained



Volatile memory loses data when power is off.



Non-volatile memory keeps data after power is off.



Volatile vs Non-Volatile

Volatile



Requires power to retain data

Non-Volatile



Retains data without power



Role of Memory



Stores program instructions



Stores temporary data



Stores configuration and permanent information



Supports processor operations during execution

SRAM vs DRAM



1. SRAM (Static RAM)



Definition

Static Random Access Memory.



Basic idea

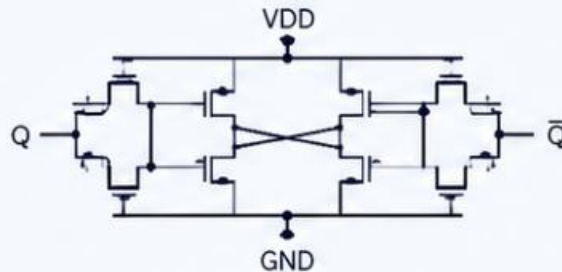
Stores data using transistor-based latch cells (typically 6 transistors per cell).



Use

Very fast memory used in CPU caches and other high-speed applications.

Built with transistor latch cells



Each cell is typically made of 6 transistors forming a latch.



2. DRAM (Dynamic RAM)



Definition

Dynamic Random Access Memory.



Basic idea

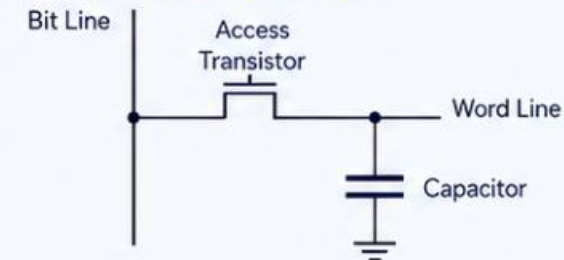
Stores data as electrical charge in capacitors.



Use

High-density main memory used in computers and mobile devices.

Built with capacitors



ROM, EPROM, and EEPROM



1. ROM



Definition

Read-Only Memory, programmed once or fixed by the manufacturer.



Use

Stores permanent firmware or basic fixed data.



2. EPROM



Definition

Erasable Programmable Read-Only Memory.



Erase method

Ultraviolet light.



Use

Can be reprogrammed after erasing.



3. EEPROM



Definition

Electrically Erasable Programmable Read-Only Memory.



Erase method

Electrical signal.



Use

Can be rewritten in-circuit and stores settings/configuration data.

Feature	ROM	EPROM	EEPROM
 Erasable	×	✓	✓
 Reprogrammable	×	✓	✓
 Erase method	Not applicable	Ultraviolet light	Electrical signal
 Practical use	Fixed firmware, basic code	Development, prototyping, BIOS	Configuration data, calibration, settings

Evolution: Increasing Flexibility



Note

These memories keep data even when power is removed.

What is a PCB?



A **PCB (Printed Circuit Board)** is the board that mechanically supports and electrically connects electronic components using copper tracks, pads, and other features.



Why PCB Matters

- Organizes and connects components
- Makes circuits compact and reliable
- Reduces wiring complexity
- Essential in nearly all embedded hardware



Study Note

Without a PCB, building compact and repeatable embedded systems would be difficult.



Anatomy of a PCB

IC Chip

The brain of the circuit (e.g., MCU)

Electronic Components

Devices placed on the board (such as ICs, resistors, capacitors, etc.)

Connector

Interface to connect external devices or power

Copper Traces

Conductive paths that carry electrical signals between components

Pads

Exposed copper areas for component leads or soldering

Resistor/Capacitor Locations

Typical placement areas for resistors and capacitors



Typical PCB Elements



Traces

Carry signals and power



Vias

Connect traces between layers



Pads

Soldering points for components



Silkscreen

Printing for labels & guides



Components

Active and passive devices on board